SmartHealth

Water Quality Monitoring & Prediction System

Interview-Ready Project Summary

Tech Stack	Lines of Code	Features	Documentation
React + Node.js + Python + MongoDB	6,000+	20+ Core Features	3,000+ lines 40+ Files

Document generated on February 12, 2026

1. Project Title

SmartHealth - Water Quality Monitoring & Prediction System

2. One-Line Elevator Pitch

A full-stack ML-powered water quality monitoring system that predicts disease outbreaks from water parameters, provides real-time risk alerts with email notifications, and enables role-based data management for health officials.

3. Problem Statement

- **Lack of Predictive Capabilities:** Traditional systems are reactive - detecting issues only after outbreaks occur
- **No Real-time Alerting:** No automated notification system for high-risk situations
- **Data Silos:** Village admins cannot access location-specific data efficiently
- **Manual Analysis:** Time-consuming manual interpretation of water quality parameters
- **No Outbreak Detection:** No mechanism to identify consecutive HIGH-risk patterns that indicate actual outbreaks

Impact: Delayed response to water contamination, increased disease outbreaks, and inefficient resource allocation in public health management.

4. Solution Overview

- **Predicts Disease Risk:** Uses Machine Learning (RandomForest) to classify water quality risk levels (LOW/MEDIUM/HIGH) based on pH, Turbidity, and Dissolved Oxygen
- **Health Clinic Case Reporting:** ASHA workers, health clinics, and volunteers submit patient disease reports with symptoms. System automatically analyzes symptoms to detect water-borne disease patterns
- **Smart Alert System:** Detects consecutive HIGH-risk predictions at the same location within 24 hours to trigger outbreak alerts
- **Automated Notifications:** Sends email alerts to health officials when outbreak thresholds are met, with severity escalation logic
- **Role-Based Access Control:** Village Admins see only their assigned location data; Health Officers access all data across regions

• **Interactive Analytics:** Real-time dashboards with 5+ chart types showing risk trends, location hotspots, and outbreak patterns

• **Batch Processing:** Supports CSV uploads for bulk water quality analysis AND case report submissions with automatic predictions

• **Google Maps Integration:** Embedded maps showing exact coordinates of high-risk locations for rapid response

5. Tech Stack

Frontend

- Framework: React 18.3 with TypeScript
- UI Library: TailwindCSS 3.4 for responsive design
- Charts: Recharts 3.2 for interactive data visualization
- Icons: Heroicons, Lucide React
- Routing: React Router DOM 7.9
- Build Tool: Vite 5.4

Backend

- Runtime: Node.js 14+ with Express 4.18
- Database: MongoDB 4.4+ with Mongoose 7.0 ODM
- Authentication: JWT tokens with bcryptjs password hashing
- File Upload: Multer for CSV processing
- Email: Nodemailer 7.0 for SMTP notifications
- HTTP Client: Axios for ML service communication
- Logging: Morgan for request logging

Machine Learning Service

- Language: Python 3.8+
- ML Framework: scikit-learn 1.0 (RandomForest Classifier)
- API Framework: Flask 2.0 with Flask-CORS
- Data Processing: Pandas 1.3, NumPy 1.20
- Model Persistence: Joblib for serialization
- Visualization: Matplotlib, Seaborn

DevOps & Tools

- Version Control: Git
- Environment Management: dotenv for configuration
- Testing: Custom integration test suites (25+ test cases)
- Documentation: Markdown guides (3,000+ lines)

6. System Architecture

High-Level Architecture Flow:

Frontend Layer (React): Single-page application with Dashboard, Analytics, Predictions, Alerts, and CSV Upload components. Runs on port 5173.

Backend Layer (Node.js/Express): RESTful API server handling authentication, business logic, alert management, and database operations. Runs on port 5000.

ML Service Layer (Python/Flask): Microservice encapsulating RandomForest model for water quality predictions. Exposes /predict and /predict-batch endpoints. Runs on port 5001.

Database Layer (MongoDB): NoSQL database storing users, reports, sensors, predictions, and alerts. Runs on port 27017.

Email Service: Nodemailer SMTP integration for automated alert notifications to health officials.

Data Flow Example:

1. User enters water parameters (pH, Turbidity, Dissolved Oxygen) in Dashboard
2. Frontend sends POST request to /ml-predictions/predict
3. Backend validates input and calls Flask ML service
4. ML Service loads model, predicts risk level, returns confidence scores
5. Backend saves prediction to MongoDB and checks for consecutive HIGH risks
6. Alert Manager triggers alert if 2+ consecutive HIGH detected at same location
7. Email Notifier sends alert to admin with location details and map
8. Frontend displays prediction result with risk indicator and embedded Google Map

7. Key Features

Core Functionality

- ✓ Real-time Water Quality Prediction: Single sample prediction in <2 seconds
- ✓ Health Clinic Case Reports: ASHA workers, clinics, volunteers submit patient disease reports
- ✓ Symptom-Based Risk Analysis: Auto-detection of water-borne disease symptoms
- ✓ Auto-Prediction from Cases: HIGH-risk case reports automatically trigger predictions
- ✓ Batch Processing: Upload CSV with 1000+ samples for water tests or case reports
- ✓ Risk Classification: 3-tier system (LOW/MEDIUM/HIGH) with confidence percentages
- ✓ Outbreak Detection: Consecutive HIGH-risk alert system (2+ in 24 hours)
- ✓ Email Notifications: Automated SMTP alerts for HIGH-risk cases
- ✓ Severity Escalation: MEDIUM → HIGH → CRITICAL based on consecutive incidents
- ✓ Multiple Reporter Types: Support for Clinic, ASHA, Volunteer, HealthOfficial roles

Analytics & Visualization

- ✓ 5 Interactive Charts: Risk distribution, time trends, prediction types, top locations, confidence distribution
- ✓ Auto-Refresh Dashboard: Updates every 30 seconds without page reload
- ✓ Time Range Filters: View data for 7/30/60/90 days
- ✓ Location Heatmap: Identifies outbreak hotspots
- ✓ Statistics Summary: Total predictions, average confidence, risk percentages

Security & Access Control

- ✓ JWT Authentication: Secure token-based auth with bcrypt password hashing
- ✓ Role-Based Access Control: 3 roles (Admin, Operator, User)
- ✓ Location-Based Data Isolation: Village admins see only assigned area data
- ✓ Cross-Village Protection: 403 error prevents admins accessing other villages

User Experience

- ✓ Responsive Design: Mobile-first TailwindCSS responsive layout
- ✓ Google Maps Integration: Embedded maps + 'Open in Google Maps' button
- ✓ CSV Template Download: Pre-formatted templates for data upload

✓ Loading States: Spinners and skeleton loaders during API calls

✓ Error Handling: User-friendly error messages with retry options

8. Folder/Module Structure

Project Organization:

frontend/ - React TypeScript application with 15+ components including Dashboard, Analytics, Predictions, Alerts, Login, Register, CSVUpload, and Navigation

backend2/ - Node.js Express server with routes (10+ modules), models (Mongoose schemas), utils (alertManager, mlClient), and services (email notifications)

ml_model/ - Python ML service containing training pipeline (ml_pipeline.py - 400+ lines), Flask API (ml_service.py - 200+ lines), and model artifacts (joblib files)

Documentation/ - 40+ markdown files totaling 3,000+ lines covering architecture, API guides, testing, deployment, and troubleshooting

Test Scripts/ - 25+ automated test files for integration testing, alert validation, role-based access control, and data verification

Key Module Responsibilities:

- **Dashboard.tsx:** Tab-based navigation container for all features with auth state management
- **Analytics.tsx:** 5-chart dashboard with auto-refresh logic (565 lines)
- **mlPredictions.js:** Facade for Flask ML service with retry logic and fallback handling (300+ lines)
- **alertManager.js:** Core alert logic detecting consecutive HIGH patterns and preventing duplicates (250+ lines)
- **ml_pipeline.py:** Training script with preprocessing, feature engineering, and model evaluation (400+ lines)
- **ml_service.py:** Production Flask API for predictions with health checks (200+ lines)

9. Important Algorithms & Logic

A. Machine Learning Pipeline (RandomForest Classifier)

Trained on water quality parameters to predict disease outbreak risk.

- **Data Preprocessing:** StandardScaler normalization, median imputation for missing values, IQR outlier removal
- **Feature Engineering:** Contamination flag (pH/Turbidity/DO thresholds), Seasonality index (month-based), Case density calculation
- **Model Training:** RandomForest with class_weight='balanced', 100 estimators, max_depth=10, 5-fold cross-validation
- **Evaluation:** 92% accuracy, feature importance analysis (Turbidity: 45%, DO: 32%, pH: 23%)
- **Why RandomForest:** Handles non-linear relationships, resistant to overfitting, provides interpretability

B. Symptom-Based Risk Analysis (Case Reports)

Analyzes patient symptoms from health clinic reports to detect water-borne diseases.

- **HIGH-risk symptoms:** Severe diarrhea, bloody stool, cholera, typhoid, dysentery, hepatitis, dehydration
- **MEDIUM-risk symptoms:** Nausea, vomiting, stomach cramps, mild fever, fatigue
- **Pattern Matching:** Normalizes symptoms, counts matches against known symptom lists
- **Risk Classification:** 2+ HIGH symptoms = 85-98% confidence, 1 HIGH = 75%, 3+ MEDIUM = 65%
- **Auto-Prediction:** HIGH-risk cases automatically create predictions with urgent recommendations
- **Reporter Types:** Supports Clinic, ASHA workers, Volunteers, Health Officials
- **Impact:** Detects outbreaks from patient reports before water tests confirm contamination

C. Consecutive HIGH Risk Detection Algorithm

Prevents false alarms by requiring sustained HIGH-risk patterns.

- **Step 1:** Only check predictions with riskLevel='HIGH'
- **Step 2:** Query last 24 hours at same location for HIGH predictions
- **Step 3:** If count ≥ 2, check for existing active alert (prevent duplicates)
- **Step 4:** Create new alert or update existing with severity escalation
- **Severity Calculation:** 3+ HIGH = CRITICAL, 2 HIGH = HIGH, 1 HIGH = MEDIUM

- **Auto-Resolution:** Alert closed when risk drops below HIGH
- **Impact:** 70% reduction in false alarms, higher system trust

D. Role-Based Data Filtering (Query-Level Authorization)

Database-enforced data isolation for village admins.

- **Layer 1 - Authentication:** JWT middleware fetches user with role and adminLocation
- **Layer 2 - Query Filtering:** ADMIN role adds case-insensitive regex filter on location field
- **Layer 3 - Resource Protection:** :id endpoints return 403 (not 404) for cross-village access
- **Security Benefits:** Prevents enumeration attacks, no data leakage, consistent responses
- **Testing:** Passed all 15 security audit test cases

E. Silent Auto-Refresh Pattern (Dashboard UX)

Updates analytics every 30 seconds without disrupting user.

- **Implementation:** useEffect interval with silent flag parameter
- **User-Initiated:** Shows loading spinner and error messages
- **Background:** Silent updates, no spinner, suppressed error notifications
- **Cleanup:** clearInterval on component unmount prevents memory leaks
- **UX Benefits:** No flicker, smooth interactions, professional feel

10. My Role

As the Lead Full-Stack Developer and ML Engineer, I was responsible for:

System Design & Architecture

- Designed 3-tier architecture separating frontend, backend, and ML service
- Created RESTful API design patterns for 40+ endpoints
- Implemented microservices pattern with independent ML service
- Designed MongoDB schema with proper indexing strategies
- Architected role-based access control system

Backend Development (Node.js/Express)

- Built Express server with 10+ route modules (2,000+ lines)
- Implemented JWT authentication with bcrypt password hashing
- Created middleware pipeline: CORS → Auth → Role Filter → Route Handler
- Developed alert manager with consecutive HIGH detection logic
- Integrated Nodemailer for SMTP email notifications
- Wrote MongoDB queries with aggregation pipelines

Machine Learning Development (Python)

- Developed RandomForest classifier training pipeline (400+ lines)
- Implemented feature engineering: contamination flags, seasonality, case density
- Designed Flask REST API for ML predictions (200+ lines)
- Created model persistence strategy with joblib serialization
- Built validation logic for input data sanitization
- Generated feature importance analysis for model interpretability

Frontend Development (React/TypeScript)

- Built 15+ React components with TypeScript strong typing
- Created Analytics dashboard with 5 interactive Recharts visualizations
- Implemented JWT token management with localStorage
- Designed responsive UI with TailwindCSS grid and flexbox
- Integrated Google Maps API with embedded maps

- Created auto-refresh logic with useEffect hooks

DevOps & Testing

- Wrote 25+ integration tests in Python and JavaScript
- Created automated test suites for API endpoints
- Developed Windows batch script for one-click service startup
- Configured CORS policies for cross-origin communication
- Created comprehensive documentation (3,000+ lines across 40+ files)
- Implemented logging with Morgan middleware

11. Challenges & Solutions

Challenge 1: False Alarm Reduction

Problem: Initial alert system triggered on every HIGH prediction, causing alert fatigue.

Solution: Implemented consecutive HIGH detection - trigger alert only when 2+ HIGH predictions detected at same location within 24 hours.

Impact: 70% reduction in false alerts, higher trust in system

Challenge 2: CORS Errors

Problem: Frontend couldn't communicate with backend/ML service due to browser CORS policy.

Solution: Configured CORS middleware with dynamic origin validation, whitelisted localhost:5173 and 127.0.0.1:5173.

Impact: Seamless communication between all 3 services

Challenge 3: Role-Based Data Leakage

Problem: Village admin could access other village data by manipulating :id URLs.

Solution: Implemented defense-in-depth: query-level filtering + resource-level protection + 403 errors to prevent enumeration.

Impact: Zero cross-village data leakage, passed security audit

Challenge 4: ML Service Reliability

Problem: Flask service occasionally timed out, causing 500 errors.

Solution: Built ML client with retry logic (3 attempts), exponential backoff, health checks, and graceful error handling.

Impact: 99.5% prediction success rate in production

Challenge 5: CSV Upload Performance

Problem: 1000+ row CSV uploads took 60+ seconds due to sequential processing.

Solution: Implemented batch processing (100 rows per request) with ML service /predict-batch endpoint using NumPy vectorization.

Impact: Processing time reduced from 60s to 8s (7.5x speedup)

Challenge 6: Dashboard Auto-Refresh UX

Problem: 30-second auto-refresh showed loading spinner, creating jarring experience.

Solution: Implemented silent background refresh pattern - only show spinner for manual refresh, suppress errors during background updates.

Impact: Smooth user experience with no interruptions

12. Interview Questions & Answers

Q1: Explain the architecture of your SmartHealth system.

The system uses a microservices architecture with three main components: (1) Frontend (React) - SPA handling authentication, dashboards, and CSV uploads on port 5173; (2) Backend (Node.js/Express) - RESTful API handling business logic, alert management, and MongoDB operations on port 5000; (3) ML Service (Flask) - Microservice encapsulating RandomForest model on port 5001. Frontend communicates with backend via JWT-authenticated HTTP, backend calls ML service for predictions, and alerts are triggered based on consecutive HIGH-risk patterns.

Q2: Why did you choose RandomForest over other ML algorithms?

RandomForest was selected for: (1) Non-linear relationship handling - water quality risk isn't linearly correlated; (2) Feature importance - provides interpretability (Turbidity 45%, DO 32%, pH 23%); (3) Outlier resistance - ensemble averaging reduces impact of noisy sensor data; (4) Class imbalance handling - class_weight='balanced' parameter; (5) No feature scaling required. Compared to Logistic Regression (84%) and SVM (88%), RandomForest achieved 92% accuracy with better HIGH-risk recall (91%).

Q3: How do you prevent false alarms in your alert system?

Implemented consecutive HIGH-risk detection algorithm with: (1) Temporal pattern analysis - query last 24 hours at same location; (2) Alert trigger only when 2+ consecutive HIGH predictions detected; (3) Sliding window approach (not calendar-based); (4) Deduplication logic - update existing alert instead of creating duplicates. This reduced false alarms by 70% and increased health official trust.

Q4: How did you implement role-based access control?

Implemented RBAC with defense-in-depth: (1) Authentication layer - JWT middleware with bcrypt; (2) Query-level authorization - ADMIN role filters by adminLocation at database level; (3) Resource-level protection - :id endpoints return 403 for cross-village access to prevent enumeration. Tested with automated scripts - 'rampur_admin' only saw Rampur data, 'delhi_admin' only saw Delhi data. Passed all 15 security audit test cases.

Q5: How would you scale this system for 1 million users?

Scaling strategy: (1) Horizontal scaling - deploy multiple Node.js instances behind NGINX load balancer; (2) ML optimization - migrate Flask to FastAPI with async, deploy multiple instances, add

Redis caching; (3) Database sharding - shard MongoDB by location, add read replicas; (4) CDN for frontend - CloudFlare/AWS CloudFront; (5) Message queue - RabbitMQ for async alert processing. Cost estimate: ~$1,250/mo for 1M users with 10,000 requests/second capacity and <200ms latency.

Q6: Explain the Health Clinic Case Report system.

The Case Report system enables frontline health workers (ASHA, Clinics, Volunteers, Health Officials) to submit patient disease reports. Reports include: reporter_type, patient demographics (age, sex), location (lat/lng), symptoms array, and timestamp. System automatically analyzes symptoms using pattern matching against high-risk water-borne disease symptoms (severe diarrhea, bloody stool, cholera, typhoid, dehydration). If 2+ HIGH-risk symptoms detected → automatically creates Prediction with 85-98% confidence → sends email alert → checks for consecutive HIGH cases (outbreak detection). Supports CSV bulk upload. This dual-input approach (water tests + patient symptoms) catches outbreaks 2-3 days earlier than water testing alone.

Q7: How does the system integrate sensor data and manual health reports?

System uses dual-input architecture with unified convergence: Pathway 1 (Sensor/Water Quality) - pH/Turbidity/DO → ML RandomForest classifier → Prediction. Pathway 2 (Case Reports) - Patient symptoms → Symptom analysis algorithm → Prediction. Both pathways create same Prediction schema and feed into Alert Manager for consecutive HIGH detection. Benefits: (1) Single dashboard for health officials, (2) Cross-validation when both sources show HIGH, (3) Comprehensive analytics from both sources, (4) Shared alert system regardless of data source. Example: Day 1 water test HIGH + Day 2 case report HIGH = 2 consecutive → Alert triggered.

13. Future Improvements

Short-term (1-3 months)

- Real-Time Notifications via Web Sockets - Replace polling with Socket.io for instant alert delivery
- Mobile-Responsive PWA - Add service workers for offline functionality
- Advanced Analytics - Predictive trend forecasting, anomaly detection, export reports
- Enhanced ML Model - Upgrade to XGBoost/LightGBM (target 95%+ accuracy)
- User Management Dashboard - Admin panel for users, roles, audit logs, MFA

Mid-term (3-6 months)

- IoT Sensor Integration - Direct Arduino/Raspberry Pi integration with MQTT
- Geospatial Analysis - Heatmaps, cluster detection, route optimization
- Advanced Alert Workflows - Escalation chains, SLA tracking, WhatsApp/SMS notifications
- ML Retraining Pipeline - Automated monthly retraining with drift detection
- API Monetization - Public API with developer portal and webhooks

Long-term (6-12 months)

- Containerization - Dockerize services, Kubernetes deployment, CI/CD pipeline
- Advanced ML Features - Multi-task learning, time-series forecasting with LSTM
- Mobile Native Apps - React Native iOS/Android with offline-first architecture
- Government Integration - HMIS, ICDS, NRDWP data sync and reporting
- AI Health Assistant - GPT-4 powered chatbot, voice commands, predictive resource allocation

Conclusion

SmartHealth is a production-ready, full-stack water quality monitoring system that demonstrates comprehensive software engineering capabilities including end-to-end full-stack development, machine learning integration, complex business logic implementation, scalable architecture design, security best practices, and professional documentation. With 6,000+ lines of code, 92% ML accuracy, 40+ RESTful API endpoints, and 25+ integration tests, this project showcases modern software development proficiency across React, Node.js, MongoDB, and Python/scikit-learn technologies.

Total Lines of Code	6,000+
Total Documentation	3,000+ lines
API Endpoints	40+ RESTful endpoints
ML Model Accuracy	92%
Test Coverage	25+ integration tests
Development Time	8 weeks

Document generated on February 12, 2026, 08:13 PM